

Function 	Description	Example
<code>abs(x)</code>	Returns the absolute value of <code>x</code> (integer).	<code>abs(-5)</code> results in <code>5</code>
<code>fabs(x)</code>	Returns the absolute value of <code>x</code> (floating-point).	<code>fabs(-5.5)</code> results in <code>5.5</code>
<code>sqrt(x)</code>	Returns the square root of <code>x</code> .	<code>sqrt(25.0)</code> results in <code>5.0</code>
<code>pow(x, y)</code>	Returns <code>x</code> raised to the power of <code>y</code> (x^y).	<code>pow(2.0, 3.0)</code> results in <code>8.0</code>
<code>ceil(x)</code>	Rounds <code>x</code> upwards to the nearest integer.	<code>ceil(3.14)</code> results in <code>4.0</code>
<code>floor(x)</code>	Rounds <code>x</code> downwards to the nearest integer.	<code>floor(3.99)</code> results in <code>3.0</code>
<code>round(x)</code>	Rounds <code>x</code> to the nearest integer (since C++11).	<code>round(2.6)</code> results in <code>3.0</code>

<code>fmod(x, y)</code>	Returns the floating-point remainder of <code>x</code> / <code>y</code> .	<code>fmod(10.5, 3.0)</code> results in <code>1.5</code>
<code>max(x, y)</code>	Returns the higher value of <code>x</code> and <code>y</code> .	<code>max(5, 10)</code> results in <code>10</code>
<code>min(x, y)</code>	Returns the lower value of <code>x</code> and <code>y</code> .	<code>min(5, 10)</code> results in <code>5</code>

Trigonometric Functions

These functions use **radians**, not degrees, for input and output.  Scribd

Function 	Description
<code>sin(x)</code>	Returns the sine of <code>x</code> .
<code>cos(x)</code>	Returns the cosine of <code>x</code> .
<code>tan(x)</code>	Returns the tangent of <code>x</code> .
<code>asin(x)</code>	Returns the inverse sine (arcsin) of <code>x</code> .
<code>acos(x)</code>	Returns the inverse cosine (arccos) of <code>x</code> .
<code>atan(x)</code>	Returns the inverse tangent (arctan) of <code>x</code> .

Function 	Description
<code>exp(x)</code>	Returns the value of E raised to the power of <code>x</code> (e^x).
<code>log(x)</code>	Returns the natural logarithm (base E) of <code>x</code> .
<code>log10(x)</code>	Returns the base-10 logarithm of <code>x</code> .

Pairs	Combines two values into a single object.	<pre>pair<int, int> p = make_pair(1, 2);</pre>
--------------	---	--

Container 	Description	Key Functions (with <code>using namespace std;</code>)
<code>vector</code>	Dynamic array.	<code>v.push_back(val);</code> <code>v.size(); sort(v.begin(), v.end());</code>
<code>set</code> , <code>unordered_set</code>	Collection of unique elements, automatically sorted (for <code>set</code>).	<code>s.insert(val);</code> <code>s.count(val);</code>
<code>map</code> , <code>unordered_map</code>	Key-value store, automatically sorted by key (for <code>map</code>).	<code>mp[key] = val;</code>
<code>stack</code>	LIFO (Last-In, First-Out) data structure.	<code>st.push(val); st.top();</code> <code>st.pop();</code>
<code>queue</code>	FIFO (First-In, First-Out) data structure.	<code>q.push(val); q.front();</code> <code>q.pop();</code>
<code>priority_queue</code>	A queue in which each element has a priority.	<code>pq.push(val); pq.top();</code> <code>pq.pop();</code>

Algorithms

`min(a, b)`, `max(a, b)`, `swap(a, b)`, `sort()`,
`reverse()`, `find()`, `count()`,
`next_permutation()`.

Numeric

`accumulate()` (sum of a range).

Bit Manipulation

Bitwise operators like `&`, `^`

Category 	Functions	Description	Example Usage
Sorting	<code>sort()</code>	Sorts elements in a range in ascending order. A custom comparator can be provided for descending or custom sorting.	<code>std::sort(v.begin(), v.end());</code>
	<code>reverse()</code>	Reverses the order of elements in a range.	<code>std::reverse(v.begin(), v.end());</code>
	<code>random_shuffle()</code>	Randomly shuffles elements (deprecated in C++17, use <code>std::shuffle</code> with a random engine).	<code>// Use std::shuffle in modern C++</code>

/cpp/algorithm.html

Searching

`find()`

Finds the first occurrence of a value in a range.

```
auto it =  
std::find(v.begin(),  
v.end(), value);
```

`binary_search()`

Checks if a sorted range contains a specific value (returns `true` or `false`).

```
bool found =  
std::binary_search(v.begin(  
, v.end(), value);
```

`lower_bound()`

Returns an iterator to the first element not less than a given value in a sorted range.

```
auto it =  
std::lower_bound(v.begin(),  
v.end(), value);
```

`upper_bound()`

Returns an iterator to the first element greater than a given value in a sorted range.

```
auto it =  
std::upper_bound(v.begin(),  
v.end(), value);
```

Min/Max & Values

`min_element()`

Returns an iterator to the minimum element in a range.

```
auto it =  
std::min_element(v.begin(),  
v.end());
```

`max_element()`

Returns an iterator to the maximum element in a range.

```
auto it =  
std::max_element(v.begin(),  
v.end());
```

`max()`, `min()`

Returns the maximum or minimum of two values.

```
int m = std::max(a, b);
```

`swap()`

Exchanges the values of two variables.

```
std::swap(a, b);
```

Manipulation

`transform()`

Applies a function to each element (or pair of elements) in a range, storing the result in a new range.

```
std::transform(v1.begin(),  
v1.end(), v2.begin(),  
result.begin(),  
std::plus<int>());
```

`fill()`

Assigns a given value to all elements in a range.

```
std::fill(v.begin(),  
v.end(), value);
```

Numeric

`accumulate()`

Calculates the sum of elements in a range, starting with an initial value (found in `<numeric>` header).

```
int sum =  
std::accumulate(v.begin(),  
v.end(), 0);
```

`count()`

Counts the number of occurrences of a specific value in a range.

```
int c =  
std::count(v.begin(),  
v.end(), value);
```

◆ `sort`

⌞ C++

```
sort(a.begin(), a.end());
```

Descendente:

⌞ C++

```
sort(a.rbegin(), a.rend());
```

Con comparator:

⌞ C++

```
sort(a.begin(), a.end(), [](int a, int b){  
    return a > b;  
});
```


obtener índice

 C++

```
int pos = lower_bound(a.begin(), a.end(), x) - a.begin();
```

4. CONTEO

count

 C++



```
count(a.begin(), a.end(), x);
```

✓ cuenta cuántas veces aparece x

 úsalo cuando:

frecuencia simple



count_if

 C++



```
count_if(a.begin(), a.end(), cond);
```

✓ cuenta bajo condición

 útil cuando:

no quieres escribir loop manual



5. BÚSQUEDA LINEAL

find

 C++



```
find(a.begin(), a.end(), x);
```

✓ busca elemento

 úsalo cuando:

n pequeño o sin ordenar



find_if

 C++



```
find_if(a.begin(), a.end(), cond);
```

✓ busca con condición



6. ELIMINAR ELEMENTOS (MUY IMPORTANTE)

remove

C++



```
auto it = remove(a.begin(), a.end(), x);  
a.erase(it, a.end());
```

✓ elimina valores



patrón obligatorio:

remove + erase



remove NO borra realmente

remove_if

C++



```
a.erase(remove_if(a.begin(), a.end(), cond), a.end());
```



✓ elimina con condición

12. ROTACIÓN (menos obvio)

rotate

 C++



```
rotate(a.begin(), a.begin()+k, a.end());
```

✓ mueve elementos

Ejemplo:

[1 2 3 4 5] → rotate k=2 → [3 4 5 1 2]



 úsalo cuando:

problemas circulares
shifts sin copiar



15. COMPARACIÓN

`equal`

⟨⟩ C++



```
equal(a.begin(), a.end(), b.begin());
```

✓ verifica igualdad



`lexicographical_compare`

⟨⟩ C++



```
lexicographical_compare(a.begin(), a.end(), b.begin(), b.end());
```

✓ compara como strings

1. eliminar duplicados

⟨⟩ C++

```
sort(a.begin(), a.end());  
a.erase(unique(a.begin(), a.end()), a.end());
```

2. coordinate compression

⌘ C++

```
vector<int> b = a;

sort(b.begin(), b.end());
b.erase(unique(b.begin(), b.end()), b.end());

for(int &x : a)
    x = lower_bound(b.begin(), b.end(), x) - b.begin();
```

📌 patrón MUY importante

3. contar ocurrencias

⌘ C++

```
int cnt = upper_bound(a.begin(), a.end(), x)
          - lower_bound(a.begin(), a.end(), x);
```

4. filtrar vector

⌘ C++

```
a.erase(remove_if(a.begin(), a.end(), cond), a.end());
```